

Algoritmos e Programação de Computadores I

Entrada e Saída pelo Console em C

Prof. Marcelo Farias

Objetivos da Aula

- Compreender as funções básicas de entrada e saída pelo console na linguagem C.
- Utilizar especificadores de formato para formatar corretamente a saída de dados.
- Realizar a leitura de diferentes tipos de dados fornecidos pelo usuário via teclado.
- Identificar limitações e cuidados ao usar funções de entrada, evitando erros comuns.

Entrada e Saída pelo Console

- O console é a interface de texto do computador que permite a troca de dados com o usuário.
- A entrada é feita pelo teclado e a saída pela tela.
- C provê funções de E/S pelo console através da biblioteca "stdio.h".

Funções de Saída

- putchar
- puts
- printf

putchar

- Declaração: *int putchar(int ch)*
- Exibe um único caractere na tela do console.
- Recebe um número inteiro que representa o caractere.
- Retorna esse mesmo número em caso de sucesso ou um valor negativo se ocorrer algum erro.

puts

- Declaração: *int puts(const char *str)*
- Exibe uma string na tela do console e adiciona automaticamente uma quebra de linha ao final.
- Recebe uma string a ser exibida.
- Retorna um número positivo em caso de sucesso ou um valor negativo se ocorrer algum erro.

printf

- Declaração: *int printf(const char *format, ...)*
- Exibe valores de variáveis formatados na tela do console.
- Recebe um especificador de formato, que indica o tipo do dado a ser exibido, e uma variável.
- Retorna o tamanho da saída em caso de sucesso ou um valor negativo se ocorrer algum erro.

Especificador de Formato do printf

%c	caractere
%s	string de caracteres
%d ou %i	inteiro com sinal
%ld ou %li %lld ou %lli	inteiro longo
%f	ponto flutuante
%e	ponto flutuante em notação científica
%u	inteiro sem sinal
%o	octal sem sinal
%x	hexadecimal sem sinal
%%	caractere %

Exibindo Caracteres

- Utilizar **%c** para exibir um caractere ASCII.
- Utilizar **%s** para exibir uma string.

Exemplo de Exibição de Caracteres

```
#include <stdio.h>
```

```
int main() {  
    char ch = 'a';  
    char str[31] = "uma string";
```

```
    printf("%c\n", ch);  
    printf("%s\n", str);
```

```
    return 0;  
}
```

Saída no console:

```
a  
uma string
```

Exibindo Números

- Utilizar **%d** ou **%i** para exibir números inteiros.
- Utilizar **%f** ou **%e** para exibir números de ponto flutuante.

Exemplo de Exibição de Números

```
#include <stdio.h>
```

```
int main() {  
    int numero = 100;  
    float moeda = 56.48;  
    double pi = 3.14159265359;
```

Saída no console:

```
100  
56.480000  
3.141593e+00
```

```
    printf("%d\n", numero);  
    printf("%f\n", moeda);  
    printf("%e\n", pi);
```

```
    return 0;
```

```
}
```

Especificador de Tamanho

- Pode-se especificar um tamanho mínimo entre % e o código de formato.
- Caso o tamanho seja menor, a saída é preenchida com espaços até atingir o tamanho mínimo.
- Espaços podem ser trocados por 0s se colocar 0 antes do tamanho mínimo.

Exemplo de Especificador de Tamanho

```
#include <stdio.h>
```

Saída no console:

```
int main() {  
    char str[31] = "uma string";  
    int numero = 100;  
    float moeda = 56.48;  
    printf("%15s\n%05d\n%12f\n",  
    str, numero, moeda);  
  
    return 0;  
}
```

```
uma string  
00100  
56.480000
```

Especificador de Precisão

- Pode-se especificar a precisão com um ponto seguido de um número após o tamanho mínimo.
- Quando aplicado em ponto flutuante, determina o número de casas decimais exibido.
- Quando aplicado em strings, determina o tamanho máximo exibido.

Exemplo de Especificador de Precisão

```
#include <stdio.h>
```

Saída no console:

```
int main() {  
    char str[31] = "uma string";  
    float moeda = 56.48;  
    printf("%5.10s, %10.1f\n", str,  
moeda);  
  
    return 0;  
}
```

uma string, 56.5

Alinhamento da Saída

- Por padrão, toda saída de printf é alinhada à direita.
- Para alinhar à esquerda o valor do tamanho mínimo deve ser negativo.

Exemplo de Saída Alinhada

```
#include <stdio.h>
```

Saída no console:

```
int main() {  
    char str[31] = "uma string";  
    float moeda = 56.48;  
    printf("%15s, %10.2f\n", str,  
moeda);  
    printf("%-15s, %-10.2f\n", str,  
moeda);  
  
    return 0;  
}
```

```
uma string,      56.48  
uma string      ,56.48
```

Funções de Entrada

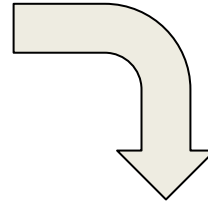
- getchar

fgets
scanf

Funcionamento do Buffer de Entrada

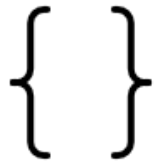


**Usuário digita 'B' e
pressiona ENTER**



Buffer de Entrada do Teclado

B	\n				
---	----	--	--	--	--



**Função de entrada lê
dados do buffer**

getchar

- Declaração: *int getchar(void)*
- Captura o próximo caractere digitado pelo usuário no console.
- Retorna o caractere lido convertido para inteiro em caso de sucesso ou um valor negativo se ocorrer um erro.

Obs.: Ela só captura o caractere depois que o usuário pressiona a tecla ENTER.

Exemplo de getchar

```
#include<stdio.h>
```

```
int main() {
```

```
    char ch;
```

```
    printf("Digite um caractere: ");
```

```
    ch = getchar();
```

```
    printf("Caractere lido: ");
```

```
    putchar(ch);
```

```
    return 0;
```

```
}
```

gets

- gets (NÃO USE)

Função antiga e insegura (removida do C11).
Causa risco de overflow de buffer.

Prefira sempre:

```
fgets(str, sizeof(str), stdin);
```

Exemplo de gets

```
#include <stdio.h>
```

```
int main() {  
    char str[31];  
    printf("Digite uma string: ");  
    /* NÃO use gets(str); */  
    fgets(str, sizeof(str), stdin);  
    printf("%s", str);  
    return 0;  
}
```


Exemplo de fgets

```
#include<stdio.h>
```

```
int main() {  
    char str[31];  
  
    printf("Digite uma string: ");  
    fgets(str, sizeof(str), stdin);  
    printf("Você digitou: ");  
    puts(str);  
  
    return 0;  
}
```

scanf

- Declaração: *int scanf(const char *format, ...)*
- Permite ler dados digitados pelo usuário no console e armazená-los em variáveis.
- Recebe um especificador de formato, que indica o tipo do dado a ser lido, e o endereço da variável (uso do operador &, exceto para strings e arrays).
- Retorna a quantidade de variáveis lidas em caso de sucesso, ou um valor negativo se ocorrer um erro.

Especificador de Formato de scanf

%c	caractere
%s	string de caracteres
%d ou %i	inteiro com sinal
%f	ponto flutuante
%e	ponto flutuante em notação científica
%u	inteiro sem sinal
%o	octal
%x	hexadecimal
%[]	busca por um conjunto de caracteres

Lendo Caracteres

- Utilizar `%c` para ler caractere ASCII.
- Utilizar `%s` para ler uma string.
- A leitura termina até encontrar um espaço em branco ou uma quebra de linha.

Exemplo de Leitura de Caracteres

```
#include <stdio.h>
```

```
int main() {  
    char ch;  
    char nome[31];  
  
    printf("Digite um caractere: ");  
    scanf("%c", &ch);  
    printf("Digite uma string: ");  
    scanf("%s", nome); // não aceita operador &  
  
    return 0;  
}
```

Lendo Números

- Utilizar **%d** ou **%i** para ler números inteiros.
- Utilizar **%f** ou **%e** para ler números de ponto flutuante.
- A leitura termina quando achar o primeiro caractere não numérico.

Exemplo de Leitura de Números

```
#include <stdio.h>
```

```
int main() {  
    int numero;  
    float moeda;  
  
    printf("Digite um número: ");  
    scanf("%d", &numero);  
    printf("Digite um valor monetário: ");  
    scanf("%f", &moeda);  
  
    return 0;  
}
```

Problemas com Leituras em C

- A função `scanf` pode deixar caracteres inválidos no buffer se a entrada não corresponder ao formato esperado.
- A função `scanf` lê strings até o primeiro espaço em branco, ignorando o restante.
- Chamadas consecutivas as funções `getchar`, `gets` e `scanf` podem ser ignoradas por conta de caracteres que ficam no buffer.

Como Evitar Problemas de Leitura?

- Limpar o buffer com a função `getchar()` após cada função de leitura.

```
while (getchar() != '\n');
```

- Usar a função `fgets()` para leitura de strings com espaços.

```
fgets(string, tamanho, stdin);
```

Mapa da aula

Aula (teoria): Aula06–08 — Entrada e saída

Laboratório guiado: Pratica04 / Pratica05

Atividade avaliativa: Atividade02

Sugestão de ritmo:

1) Conceitos nos slides 2) Prática no lab 3) Atividade sem 'receita'

Para estudar no livro

Livro-base: ANTONELLO, Ricardo. Linguagem C. 1. ed.

Leitura indicada: Cap. 4.7 e 4.8 (p. 33–38)

Exercícios do livro: Exercícios 4.13 (E/S)

Revise o capítulo após a aula e antes da prática.

Desafio (8 min)

Leia nome completo (espaços) e idade; imprima formatado. Use fgets + scanf com limpeza de buffer.

Trabalhe em dupla (instrução por pares).

Ao final, uma dupla compartilha a solução com a turma.

Quiz rápido (3 perguntas)

- 1) Por que gets é perigosa?
- 2) scanf("%s") lê nome composto?
- 3) Como limpar o buffer após scanf?

Respostas no quadro / Mentimeter / Kahoot.

Referências

- SCHILDT, Herbert. C: Completo e Total. 3. ed. Pearson Makron Books, 1997.
- ANTONELLO, Ricardo. Linguagem C. 1. ed. Livro Digital, 2020.